Practice 13. Sorting

[CSE2010] Data Structures

Department of Data Science

Practice 10 Solution (Python)

- Implement the graph class
- Member variables
  - nVertices
  - nEdges,
  - Neighbors
- Methods
  - build(self, n, m, data)
  - isAdjacent(self, u, v)
  - getNeighbors(self, u)

```python
class Graph:
  def __init__(self):
    self.nVertices = -1
    self.nEdges = -1
    self.neighbors = None

  def build(self, n, m, data):
    self.nVertices = n
    self.nEdges = m
    self.neighbors = [list() for _ in range(n)]
    for i, (u, v, w) in enumerate(data):
      assert(0 <= u < n)
      assert(0 <= v < n)
      self.neighbors[u].append((v, w))

  def isAdjacent(self, u, v):
    assert(0 <= u < self.nVertices)
    assert(0 <= v < self.nVertices)
    for nbr, w in self.neighbors[u]:
      if nbr == v:
        return True
    return False

  def getNeighbors(self, u):
    assert(0 <= u < self.nVertices)
    return [v for v, w in self.neighbors[u]]
```

Practice 10 Solution (C++)

```cpp
class Graph {
public:
  int nVertices;
  int nEdges;
  pair<int, int>** neighbors;
  int* nNeighbors;
  int* distance;
  int* predecessor;
  Graph():neighbors(nullptr), nNeighbors(nullptr), distance(nullptr), predecessor(nullptr) {}
  ~Graph();
  void printGraph();
  void build(int, int, vector<tuple<int, int, int>>&);
  bool isAdjacent(int, int);
  vector<int> getNeighbors(int);
  vector<int> breadthFirstSearch(int);
  vector<int> depthFirstSearch(int);
  bool isReachable(int, int);
  bool runDijkstra(int, int, vector<int>&);
  bool topologicalSort(vector<int>&);
};

Graph::~Graph() {
  if (neighbors != nullptr) {
    for (int i = 0; i < nVertices; ++i) {
      if (neighbors[i] != nullptr)
        delete neighbors[i];
    }
    neighbors = nullptr;
  }
  if (nNeighbors != nullptr) {
    delete nNeighbors;
    nNeighbors = nullptr;
  }
  if (distance != nullptr) {
    delete distance;
    distance = nullptr;
  }
  if (predecessor != nullptr) {
    delete predecessor;
    predecessor = nullptr;
  }
```

```cpp
void Graph::build(int n, int m, vector<tuple<int, int, int>>& data) {
  int u, v, wgt;
  nVertices = n;
  nEdges = m;
  nNeighbors = new int[n]();
  distance = new int[n];
  predecessor = new int[n];
  vector<tuple<int, int, int>>::iterator it;
  for (it = data.begin(); it != data.end(); ++it) {
    tie(u, v, wgt) = *it;
    assert(u >= 0 && u < nVertices);
    assert(v >= 0 && v < nVertices);
    ++nNeighbors[u];
  }
  neighbors = new pair<int, int>*[nVertices];
  for (int i = 0; i < nVertices; ++i) {
    if (nNeighbors[i] > 0)
      neighbors[i] = new pair<int, int>[nNeighbors[i]];
    else
      neighbors[i] = nullptr;
  }
  memset(nNeighbors, 0, sizeof(int) * nVertices);
  for (it = data.begin(); it != data.end(); ++it) {
    tie(u, v, wgt) = *it;
    neighbors[u][nNeighbors[u]++] = make_pair(v, wgt);
  }
}

bool Graph::isAdjacent(int u, int v) {
  assert(u >= 0 && u < nVertices);
  assert(v >= 0 && v < nVertices);
  for (int j = 0; j < nNeighbors[u]; ++j) {
    if (neighbors[u][j].first == v)
      return true;
  }
  return false;
}

vector<int> Graph::getNeighbors(int u) {
  assert(u >= 0 && u < nVertices);
  vector<int> nbrs(nNeighbors[u]);
  for (int j = 0; j < nNeighbors[u]; ++j)
    nbrs[j] = neighbors[u][j].first;
  return nbrs;
}
```

Practice 11 Solution (C++)

- Method topologicalSort returns **whether the sort exists** and **the topological sort** if so.

- Use std::queue<int> as the queue implementation.

```
    case TOPOLOGICAL_SORT:
      if (g.topologicalSort(p)) {
        for (vector<int>::iterator it = p.begin(); it != p.end(); ++it){
          if (it != p.begin()) outFile<<" ";
            outFile<<*it;
        }
        outFile<<endl;
      }
      else {
        outFile<<"N"<<endl;
      }
      break;
```

```
bool Graph::topologicalSort(vector<int>& out) {
  out.clear();
  queue<int> q;
  int cnt = 0;
  int* inCnt = new int[nVertices]();
  for (int u = 0; u < nVertices; ++u) {
    for (int i = 0; i < nNeighbors[u]; ++i) {
      ++inCnt[neighbors[u][i].first];
    }
  }
  for (int u = 0; u < nVertices; ++u) {
    if (!inCnt[u])
      q.push(u);
  }
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    out.push_back(u);
    for (int i = 0; i < nNeighbors[u]; ++i) {
      int v = neighbors[u][i].first;
      --inCnt[v];
      if (!inCnt[v])
        q.push(v);
    }
    ++cnt;
  }
  delete inCnt;
  return (cnt == nVertices);
}
```

Practice 11 Solution (C++)

- Method runDijkstra returns **whether the shortest path exists** and **the path** if so.

- Use a min heap

```cpp
    case SHORTEST_PATH:
      if (!(iss >> u >> v)) {
        cerr<<"SHORTEST_PATH: invalid input"<<endl;
        exit(1);
      }
      if (g.runDijkstra(u, v, p)) {
        // Write path into output file
        for (vector<int>::iterator it = p.begin(); it != p.end(); ++it){
          if (it != p.begin()) outFile<<" ";
          outFile<<*it;
        }
        outFile<<endl;
      }
      else {
        outFile<<"N"<<endl;
      }
      break;
```

```cpp
bool Graph::runDijkstra(int s, int t, vector<int>& output) {
  fill(distance, distance + nVertices, INT_MAX);
  memset(predecessor, -1, sizeof(int) * nVertices);
  assert(s >= 0 && s < nVertices);
  assert(t >= 0 && t < nVertices);
  distance[s] = 0;
  // Build a min priority queue Q with keys of distance[v] for all v
  MinHeap heap(nVertices);
  heap.build(distance, nVertices);
  while (!heap.isEmpty()) { // While Q is not empty
    Element curr = heap.deleteMinimum(); // Delete the min from Q
    int u = curr.id, v, wgt;
    for (int i = 0; i < nNeighbors[u]; ++ i) {
      tie(v, wgt) = neighbors[u][i];
      if (distance[u] < (INT_MAX - wgt) && distance[u] + wgt < distance[v]) {
        distance[v] = distance[u] + wgt;
        heap.decreaseKey(v, distance[v]);
        predecessor[v] = u;
      }
    }
  }
  int u = t;
  while (u != s) {
    output.push_back(u);
    u = predecessor[u];
    if (u < 0) {
      output.clear();
      return false;
    }
  }
  output.push_back(s);
  reverse(output.begin(), output.end());
  return true;
}
```

TODO

- Implement **merge sort** that sorts a sequence of integers in the **descending** order
- Implement **quicksort** that sorts a sequence of integers in the **descending** order

Input and Output

- Definition of input
  - **M<space>[number_of_integers]**
    [int]<space>[int], …, <space>[int]
    - Run merge sort, and write a sorted sequence into output file.
  - **Q<space>[number_of_integers]**
    [int]<space>[int], …, <space>[int]
    - Run quicksort, and write a sorted sequence into output file.
- Input file example

```
M 1
0
M 5
2 5 4 1 3
M 10
103 2 213 153 2 10 25 8 1 10
Q 1
0
Q 5
2 5 4 1 3
Q 10
103 2 213 153 2 10 25 8 1 10
```

Output file example

```
0
5 4 3 2 1
213 153 103 25 10 10 8 2 2 1
0
5 4 3 2 1
213 153 103 25 10 10 8 2 2 1
```

Running Example

- You can start from the scratch, or the skeleton code.

```
[hjkim@localhost sorting]$ cat input1.txt
M 1
0
M 5
2 5 4 1 3
M 10
103 2 213 153 2 10 25 8 1 10
Q 1
0
Q 5
2 5 4 1 3
Q 10
103 2 213 153 2 10 25 8 1 10
[hjkim@localhost sorting]$ python sorting.py input1.txt output1.txt
[hjkim@localhost sorting]$ cat output1.txt
0
5 4 3 2 1
213 153 103 25 10 10 8 2 2 1
0
5 4 3 2 1
213 153 103 25 10 10 8 2 2 1
```

Submission Guideline

- Submission: **source code, an input file, an output file**
  - Where: Assignments in LMS
  - Deadline: **23:59, June 2nd (Sunday)**